CYLAB SECURITY ACADEMY

Challenge Write-Up Template

Instructions: Copy this template for every challenge you submit. Fill in every section — a write-up that only shows the final flag without explanation will not receive full credit. Delete these italicized instruction lines before submitting.

Challenge Information

Student Name	Keyzia Joy Pascua
Section / Class	BSIT 3 - Networking
Challenge Name	Password Profiler
Category	General Skills
Points / Difficulty	Easy
Date Solved	August 11, 2026
Flag Format Given	picoCTF{...}

1. Challenge Description

This challenge is about OSINT-driven password cracking rather than brute force in the dark. You're given a SHA-1 hash of someone's password plus a set of personal details about them (name, nickname, birthdate, partner, child), and a script that checks candidate passwords against that hash. The point is to turn personal information into a realistic password list - the same way an attacker might profile a real target - and then let the script find the match.

---

2. Initial Analysis / Reconnaissance

Before touching any tooling, get organized: put the downloaded files in one folder and read every file before writing or running anything. Reading check_password.py first tells you exactly what it expects - a file called passwords.txt in the same directory, one password per line, each checked with SHA-1. Reading userinfo.txt tells you what raw material you have to work with (name, nickname, birthdate, partner, child). Together, this shows the challenge is really about generating the right wordlist, not about the cracking script itself.

3. Tools and Commands Used

Tool / Command	What it was used for	Result / Output
mv + cd	Move the downloaded files into a dedicated working folder and inspect them	userinfo.txt, hash.txt, and check_password.py confirmed in ~/password_profiler
cat userinfo.txt / hash.txt / check_password.py	Read the victim's personal details, the target SHA-1 hash, and how the checker script verifies a guess	OSINT-style profile (name, nickname, birthdate, partner, child) + a 40-character SHA-1 hash + a script that hashes each line of passwords.txt and compares it to hash.txt
sudo apt install cupp -y	Install CUPP (Common User Passwords Profiler), a tool that builds a custom wordlist from personal details about a target	cupp installed and ready to run
cupp -i	Run CUPP interactively and feed it the victim's profile to generate a targeted password dictionary	alice.txt saved, containing 28,888 candidate passwords
mv alice.txt passwords.txt && python3 check_password.py	Point the provided cracking script at the CUPP-generated wordlist and let it brute-force the hash	Password found: picoCTF{Aj_15901990}

4. Solution Walkthrough

Step 1

Start by putting the challenge files somewhere predictable and reading each one. Move userinfo.txt, hash.txt, and check_password.py into a working folder, then cat each file. userinfo.txt gives personal details about the target (name, nickname, birthdate, partner's name, child's name) - exactly the kind of information a real OSINT-based password guesser would use. hash.txt holds a single SHA-1 hash, and check_password.py shows the checking logic: it reads passwords.txt line by line and compares each SHA-1 hash to the one in hash.txt.

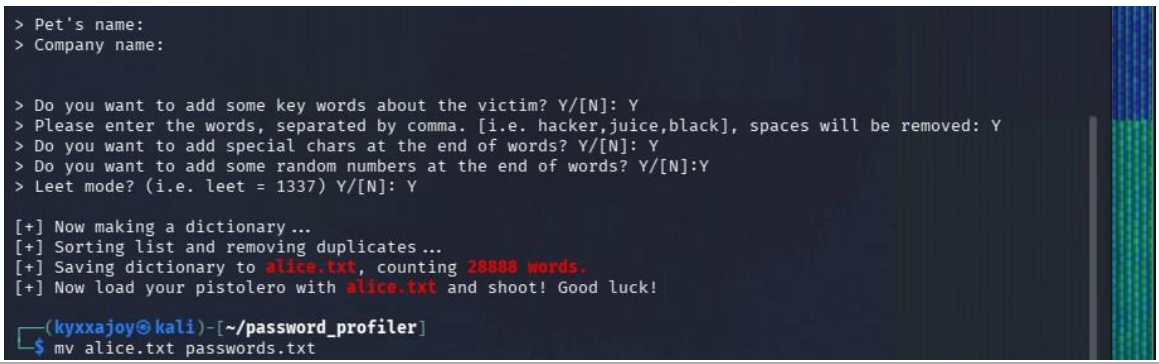
```
-rw-rw-r--  1 kyxxajoy kyxxajoy  731 Aug 16 14:47 check_password.py
-rw-rw-r--  1 kyxxajoy kyxxajoy   41 Aug 16 14:47 hash.txt
-rw-rw-r--  1 kyxxajoy kyxxajoy  113 Aug 16 14:47 userinfo.txt
First Name: Alice
Surname: Johnson
Nickname: AJ
Birthdate: 15-07-1990
Partner's Name: Bob
Child's Name: Charlie

968c2349040273dd57dc4be7e238c5ac200ceac5
#!/usr/bin/env python3
import hashlib

HASH_FILE = "hash.txt"
WORDLIST_FILE = "passwords.txt" # wordlist that was generated using CUPP

def load_hash():
    with open(HASH_FILE, "r") as f:
        return f.read().strip()

def crack_password(target_hash):
    with open(WORDLIST_FILE, "r", encoding="utf-8", errors="ignore") as f:
        for password in f:
            password = password.strip()
```

Step 2

Since the checker script expects a file named passwords.txt built from candidate passwords, the next step is generating that list. CUPP (Common User Passwords Profiler) is purpose-built for this: it takes personal details about a target and produces a wordlist based on common patterns people use in real passwords (names, nicknames, dates, leetspeak substitutions, etc). Install it with sudo apt install cupp -y.

Step 3

Run cupp -i to start the interactive profiler, then answer its prompts using the details from userinfo.txt: First Name Alice, Surname Johnson, Nickname AJ, Birthdate 15071990, Partner's name Bob, Child's name Charlie. Leave fields you don't have (partner/child nicknames and birthdates, pet, company) blank rather than guessing. When asked about extra options - key words, special characters, trailing numbers, leet-speak substitutions - say yes to each, since these are exactly the kinds of variations real people use when turning personal facts into passwords. CUPP then builds and saves a dictionary (here, alice.txt with 28,888 candidate passwords) covering the many likely combinations of that information.



Additional steps (add as needed)

With a targeted wordlist generated, rename it to match what the checker script expects (passwords.txt) and run the script: mv alice.txt passwords.txt then python3 check_password.py. The script hashes every candidate in the wordlist with SHA-1 and compares it against the target hash from hash.txt; because the wordlist was

built specifically from the victim's real details rather than a generic dictionary, it finds a match almost immediately and prints the flag: picoCTF{Aj_15901990}.

---

5. Flag

```
picoCTF{Aj_15901990}
```

6. Key Concept / What I Learned

The key concept here is OSINT-informed password cracking: rather than brute-forcing every possible string, you narrow the search space dramatically by building a wordlist from what you know about the target (names, nicknames, dates, family members) and common ways people mutate that information into passwords (capitalization, leet-speak, trailing numbers/symbols). Tools like CUPP automate that process, and this pattern - profile first, then generate a targeted list, then crack - is exactly how real-world password attacks against individuals often work.

---

7. References

CUPP (Common User Passwords Profiler) documentation/GitHub (github.com/Mebus/cupp) - used to understand its interactive prompts and dictionary-generation options.
